

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	SECTION: CPE21S1
DATE PERFORMED: Aug 27 2026	DATE SUBMITTED: Aug 27 2026
NAME: Mendoza, Roy Gabriel I.	INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. Exam	Prelim Examination
--------------------------	---------------------------

1. PROCEDURE OUTPUT

What procedures did you perform?

CODE IS DONE from 1030 to 1249pm today

PSEUDOCODE:
START ASK user for input (as a string) READ the input from the user INITIALIZE stack (array-based) with capacity = length of input INITIALIZE queue (array-based or linked-list) with capacity = length of input FOR each character in the input: PUSH character onto stack ENQUEUE character into queue WHILE stack is not empty AND queue is not empty: POP from stack DEQUEUE from queue IF popped value and dequeued value is not equal: PRINT "Not Palindrome" STOP PRINT "Palindrome" END

CODE:

STACK - ARRAY

```
3 > //...
4
5 #ifndef UNTITLED1_ARRSTACK_H
6 #define UNTITLED1_ARRSTACK_H
7 > #include ...
8
9
10
11 ✓ //=====
12 // ARRAY-BASED STACK
13 ✓ /*
14 void push(); check
15 void pop(); check
16 void Top(); check
17 bool isEmpty(); check
18 bool isFull(); check
19 void prinAll(); check
20 */
21 ✓ //=====
22 // A. STACK USING ARRAYS
23
24 template <typename T>
25 ✓ class arrStack {
26 private:
27     T* stackArray;
28     int cap;
29     int top;
30
31 public:
32 ✓     arrStack(int size = 10) {
33         // setting the maximum size of the stack
34         cap = size;
35         stackArray = new T[cap]; // Array that will store the symbols
36         top = -1; // Indexing the top element
37     }
38
```

```

39  ▾ //-----
40    // Check if the stack is empty
41  ▾ bool isEmpty(){
42      return (top == -1);
43  }
44  ▾ //-----
45    // Checking if the stack is full
46  ▾ bool isFull(){
47      return (top == cap - 1);
48  }
49  ▾ //-----
50    // Display the top element
51  ▾ T Top()const{
52  ▾     if (isEmpty()){
53         std::cout << "Stack is Empty.\n";
54         return T();
55     }
56     return stackArray[top];
57 }
58  ▾ //-----
59    // Push a character onto the stack
60  ▾ void push(T value){
61      // Prevent overflow
62  ▾     if (isFull()){
63         std::cout << "Stack Overflow!\n";
64         return;
65     }
66     // Move top then insert the value
67     stackArray[++top] = value;
68 }
69  ▾ //-----

```

```

70 // Remove the top/last inputted elemetn
71 T pop(){
72     // Prevent underflow
73     if (isEmpty()){
74         std::cout << "Stack Underflow!\n";
75         return T();
76     }
77
78     // Return current top then decrease top
79     return stackArray[top--];
80 }
81 //-----
82 // Display all elements
83 void displayAll(){
84     if (isEmpty()){
85         std::cout << "Stack is Empty.\n";
86         return;
87     }
88     std::cout << "Stack Contents (Top to Bottom)\n";
89
90     for (int i = top; i >= 0; i--){
91         std::cout << "|" << stackArray[i] << "|" << std::endl;
92     }
93 }
94 //-----
95 // Reset the stack
96 void clear() {
97     top = -1;
98 }
99

```

```

100 //-----
101 //Destructor
102 ~arrStack() {
103     // Release the dynamically allocated array
104     delete[] stackArray;
105 }
106 };
107
108 #endif // UNTITLED1_ARRSTACK_H

```

QUEUE - LL

```
1  > //...
4
5  #ifndef UNTITLED1_QUEUEARR_H
6  #define UNTITLED1_QUEUEARR_H
7  #include <iostream>
8  #include <string>
9
10 //QUEUE CLASs
11 //circular array - based on the module example
12 template <typename T>
13 class queueArr {
14     private:
15         T* q_array; // pointer for the dynamic allocation of array
16         int q_size; // current number of elements stored
17         int q_capacity; // maximum number of elements
18         int q_front; // index the front element of the queue
19         int q_back; // index the back element
20
21     public:
22         queueArr(int capacity = 10) {
23             q_capacity = capacity;
24             q_array = new T[q_capacity];
25             q_size = 0;
26             q_front = 0;
27             q_back = -1;
28             std::cout<<"Queue is created"<<std::endl;
29             std::cout<<"Constructor was Called"<<std::endl;
30         }
31
32         // checker if queue is empty
33         bool empty() const {
34             return (q_size == 0);
35         }
36
37         //checker for the number of element in the queue
38         int size() const {
39             return q_size;
40         }
```

```

42 //removing of all the elements in the queue
43 void clear() { //resetting the size, front, back, but does not affect the heap nor capacity
44     q_size = 0;
45     q_front = 0;
46     q_back = -1;
47 }
48
49 //showing of the front element
50 T front() const {
51     if (empty()) {
52         std::cout<<"Queue is empty"<<std::endl;
53         return T();
54     }
55     return q_array[q_front];
56 }
57
58 //returns the last element in the queue
59 T back() const {
60     if (empty()) {
61         std::cout<<"Queue is empty"<<std::endl;
62         return T();
63     }
64     return q_array[q_back];
65 }
66
67 // add a new element at the back of the last element
68 void enqueue(T item) {
69     if(q_size == q_capacity) {
70         std::cout<<"Queue is full"<<std::endl;
71         return;
72     }
73     q_back = q_back + 1%q_capacity;
74     q_array[q_back] = item;
75     q_size++;
76 }

```

```

77
78 //removing of the front element
79 → T dequeue() {
80     if (empty()) {
81         std::cout<<"Queue is empty"<<std::endl;
82         return T();
83     }
84     T delValue = q_array[q_front];
85     q_front = q_front + 1%q_capacity;
86     q_size--;
87     return delValue;
88 }
89
90 //copy constructor
91 → queueArr(const queueArr& other) { //creates a new queue copying the old queue
92     q_capacity = other.q_capacity; //copy capacity
93     q_size = other.q_size; //copy the size
94     q_front = other.q_front; // copy the front index
95     q_back = other.q_back; // copy the back index
96     q_array = new T[q_capacity]; //creates a new array
97
98     for (int i = 0; i < q_capacity; i++) { //copy the entire array
99         q_array[i] = other.q_array[i]; //copy each array positions
100     }
101     std::cout<<"Queue is created"<<std::endl;
102     std::cout<<"Copy Constructor was Called"<<std::endl;
103 }
104

```

```

105 //copy assignment operator
106 → queueArr& operator=(const queueArr& other) {
107     if (this == &other) {
108         return *this;
109     }
110     delete[] q_array;
111     q_capacity = other.q_capacity;
112     q_size = other.q_size;
113     q_front = other.q_front;
114     q_back = other.q_back;
115     q_array = new T[q_capacity];
116
117     //copying the entire array
118     for (int i = 0; i < q_size; i++) {
119         q_array[i] = other.q_array[i];
120     }
121
122     return *this;
123 }
124
125 //destructor
126 → ~queueArr() {
127     delete [] q_array;
128     std::cout<<"Queue is deleted"<<std::endl;
129     std::cout<<"Destructor was called Successfully";
130 }
131 };
132 #endif //UNTITLED1_QUEUEARR_H

```


MAIN

```
1 //  
2 // Created by royga on 8/27/2026.  
3 //  
4 #include <iostream>  
5 #include <string>  
6 #include "arrStack.h"  
7 #include "queueLL.h"  
8 //  
9 //ALGORITHM:  
10 //ASK THE USER FOR THE INPUT THAT'S NEED TO BE CHECKED  
11 //MAKE THE NUMBERS LIKE A STRING  
12 //CREATE A STACK AND ARRAY  
13 //PUSH EACH DIGIT ON THE STACK, AND ENQUEUE INTO QUEUE  
14 //COMPARE THEM SIMULTANEDUSLY LIKE SINCE STACK IS FILO AND QUEUE IS FIFO, WE CAN JUST USE THIS SIMULTANEOUS REMOVAL TO COMPARE  
15 //OUTPUT IF TRUE PALENDROME OR NOT  
16 //  
17 // Function to check palindrome using stack + queue  
18 // since I will be combining both, instead of using another header, I can just put it in here in the main  
19 bool isPalindrome(const std::string& str) {  
20     arrStack<char> s(str.size()); // stack (array-based)  
21     queueArr<char> q(capacity: str.size()); // queue (array-based)  
22  
23     // push/enqueue each character<- combine them(thanks twain for the idea)  
24     for (char c : str) {  
25         s.push(c);  
26         q.enqueue(c);  
27     }  
28  
29     // compare stack pop vs queue dequeue  
30     while (!s.isEmpty() && !q.empty()) {  
31         char fromStack = s.pop();  
32         char fromQueue = q.dequeue();  
33         if (fromStack != fromQueue)  
34             return false;  
35     }  
36     return true;  
37 }  
38  
39 int main() {  
40     std::string input;  
41     std::cout << "Enter a number: ";  
42     std::cin >> input;  
43  
44     if (isPalindrome(input))  
45         std::cout << "\n\n" << input << " is a Palindrome\n";  
46     else  
47         std::cout << "\n\n" << input << " is NOT a Palindrome\n";  
48  
49     return 0;  
50 }
```

OUTPUT:

```
"C:\Users\royga\CLionProjects\untitled1\PRELIM EXAM FIELS\main.exe"  
Enter a number: 2513234  
Queue is created  
Constructor was Called  
Queue is deleted  
Destructor was called Successfully  
  
2513234 is NOT a Palindrome  
  
Process finished with exit code 0
```

```
"C:\Users\royga\CLionProjects\untitled1\PRELIM EXAM FIELS\main.exe"  
Enter a number: 10201  
Queue is created  
Constructor was Called  
Queue is deleted  
Destructor was called Successfully  
  
10201 is a Palindrome  
  
Process finished with exit code 0
```

CODE ANALYSIS:

This Palindrome Exam Act requires us to determine if the string/characters that we have inputted, if reverse outputted, would be the same as the input. In this act, I used 2 structures, the stack and queue.

In **queue**, I implemented, using linked lists, where when a user inputs a number, the program converts the string input to a cutted string producing like a single value integers. Then, in **stack**, I used an array. Each digit is pushed into stack, and enqueued in queue.

I then have a comparison scenario, where the program repeatedly pops the stack and dequeue from the queue, checking if the values are equal at each step. If any mismatch happened, it then print outs "not a palindrome" else, "it is a palindrome". Since the structures of stack and queue are completely different ,LIFO and FIFO, it can compare them at each step and check both array and linked list if the string is a palindrome.

Overall, this activity requires a lot of time if you are going to do it from scratch, like not use any libraries but your own made functions. And since some parts are already done on the past activities, I just copied the implementation, and change the data types, and we have it(except for the main).

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.